Week 4 Lecture 2

Assignment 2 Discussion

Software Engineering ?

- Software engineering is the systematic application of engineering principles, methods and practices to the design, development, testing, deployment and maintenance of software systems.

Is software engineering considered an engineering discipline rather than simply programming discipline ?

Yes, sometimes. Because it applies structured processes, design methods, quality assurance, risk management, documentation, teamwork and lifecycle thinking to produce dependable software, whereas programming is primarily the activity of implementing code.

Do many software projects fail even when programmers are technically capable?

- Yes, but why ?

Because of unclear requirements, poor planning, weak communication, unmanaged scope, inadequate architecture, insufficient testing, unrealistic schedules or poor stakeholder involvement, even when coding skill is strong.

How to distinguish high quality software from software that just works ?

- Certain characteristics
  - First it works
  - But it is also reliable, maintainable, usable, secure, efficient, testable, understandable and adaptable to change.

What is maintainability of professional software ?

- maintainability is considered as one of the most important characteristics of professional software.
- Because software spends much of its life being corrected, adapted and extended.
- Maintainable software reduces the time, cost and risk of future changes.

Importance of understanding the problem before writing code

- software engineers need to spend time understanding the problem before writing code.
- Because coding the wrong solution efficiently is still failure.
- Understanding users, goals, constraints and business rules first reduces rework and helps ensure the software solves the actual problem.
- This is called **Requirements engineering**

Functional vs. non-functional requirements.

- **Functional** requirements describe what the system must do, such as enrol a student or generate a transcript.
- **Non-functional** requirements describe qualities or constraints, such as performance, security, usability or reliability.

What are stakeholders ?

- stakeholders important during software development.
- Stakeholders provide needs, priorities, constraints, feedback and acceptance criteria.
- Without them, engineers may build technically correct software that does not meet organisational or user needs.

What are business rules ?

- Business rules are domain-specific policies or constraints, such as GPA ranges or course-capacity limits.
- The software development design process should enforce them consistently to preserve valid system state and correct organisational behaviour.

Addressing Changing Requirements

- Software requirements often change during development.
- How should software engineers respond to changing requirements?
  - They should assess the impact, clarify and prioritise the change with stakeholders, update requirements/design/tests, and implement it in a controlled way rather than resisting all change or changing code informally.

What is problem decomposition ?

- Problem decomposition is breaking a large or complex problem into smaller, manageable subproblems or components that can be understood, designed, implemented and tested separately.

How to do problem decomposition ?

- software engineers divide large systems into smaller components.
- Why ?
  - To reduce complexity, improve understanding, localise change, enable reuse, support testing and allow different developers to work on separate parts.

What is the benefit of problem decomposition ?

- Problem decomposition can improve software testing.
- It can allow components to be tested independently with focused test cases, making failures easier to isolate and diagnose.

More benefits of problem decomposition

- Improves teamwork.
- How
  - Well-defined modules/components create clear ownership and interfaces, allowing team members to work in parallel with less interference.
- Modular software development allows better risk management.
- How ?
  - Changes cause risk, and any changes can have unpredictable effects, code becomes harder to understand and test, duplication increases, defects are harder to isolate and team collaboration becomes more difficult.

How to do modular software development ?

- Object oriented software design.
- Object-oriented vs. function-based software
- Aren’t they both modular software development approaches?
- Which is better ?
- Why?

Identifying objects within a business problem ?

- How do software engineers identify objects within a business problem?
- Analyse important domain concepts, often starting from significant nouns, then ask whether each concept has meaningful identity, state, behaviour or responsibility that the system must manage.

Objects and classes

- What is an object ?
  - An object is a specific instance of a class that has its own state and can perform behaviours defined by the class.
- difference between an object and a class.
  - A class is a blueprint or type definition describing common attributes and behaviours; an object is one concrete instance created from that class.

Designing better classes and objects

- Every object needs to have clear responsibilities. Why ?
  - improve cohesion, make behaviour easier to locate and test, and prevent classes from becoming overloaded with unrelated tasks.
- Objects need to collaborate to solve business problems . How ?
  - Objects collaborate through relationships and method calls, with each object contributing its own responsibility to a larger workflow

Major phases of the Software Development Lifecycle (SDLC)

- Typical phases include
  - requirements analysis,
  - design,
  - implementation,
  - testing,
  - deployment
  - maintenance/evolution
- Iterative methods may revisit these phases repeatedly (Waterfall, Agile .. Software development methodology)

Importance of Testing

- Testing needs to be performed throughout software development rather than only at the end.
- Why ?
  - Early and continuous testing finds defects sooner, validates assumptions, reduces the cost of fixes and provides regression protection as the system evolves.

Why do you need to maintain software?

- Maintenance of software is important and it is often the longest phase of a software system's life.
- After release, software must continue to fix defects, adapt to new requirements and environments, improve performance and remain secure, often for many years.

Why to do documentation ?

- Project documentation is also important and needs to be developed throughout the project rather than after implementation.
- Continuous documentation stays aligned with decisions as they are made, reduces forgotten rationale, supports collaboration and makes final documentation more accurate.

How to manage risks in software projects ?

- Using a SDLC methodology can reduce risks.
- How?
  - By introducing structured checkpoints for understanding requirements, reviewing design, testing quality, managing changes and validating deliverables before release.

Assignment 2 – Case Study Discussion

- Assignment 2 reflect professional software engineering practice. How ?
  - It requires students to analyse a realistic domain, design classes/UML, implement modular OO software, test it, document it, use version control (Git) and explain design decisions orally—activities that mirror professional development practice.

Software Engineering Practice in Generative AI Era

- If AI Can Write Code, Why Do We Need Software Engineers?
  - Can AI generate Python code?
  - Can AI explain and debug code?
  - Can AI build complete features?
  - What remains for the human engineer?

The Changing Nature of Software Work

- **Traditional:** programmer writes code
- **Emerging:** engineer understands, specifies, designs, builds, tests, reviews, integrates and takes responsibility
- **Emerging:** Takes Assistance from GenAI **.** How ?
  - To Explain code, Generate functions and classes, Suggest tests, Debug errors,
  - Review and refactor, Document and plan Modify repository-level work

The New Educational Question

Old thinking : Can the student write code?

New thinking: Can the student engineer trustworthy software while intelligently using AI?

Programming : Variables and data structures, Conditions and loops, Functions, Classes, Algorithms

Engineering : Specification, Design, Implementation, Verification, Maintenance, Evolution

Programming: code-focused and implementation-oriented

Software engineering: system-focused and lifecycle-oriented

Programming asks: Does it run?

Engineering asks: Is it correct, maintainable and appropriate?

A Tiny Appointment Program

```
appointments = []
add_appointment(patient, doctor, time)
```

- Question: Is this software engineering?
- Correct? Reliable? Maintainable? Secure? Testable?
- Understandable/Interpretable ? Scalable/Extensible/Evolvable?
- SDLC: Problem -> Requirements -> Design -> Implementation -> Testing -> Deployment -> Maintenance

First two steps

- **Step 1: Problem Description**
  - What is the problem?
  - What currently happens?
  - What should improve?
  - What constraints exist?
  - What information is needed?
  - What could go wrong?
- **Step 2: Requirements**
  - What must the system do?
  - What constraints apply?
  - What quality is expected?
  - How should it behave?
  - What do stakeholders need?

Step 3, 4 and 5

- Step 3: Design
  - Components
  - Responsibilities
  - Relationships
  - Data structures
  - Interfaces
  - Architecture
- Step 4: Implementation
  - Convert design into executable software
  - Primary language: Python
- **Step 5: Testing**
  - Does the software behave as required?
  - Not merely: Does the program start?

Step 6: Maintenance and Evolution

New requirements

Defects

New users

Security issues

Technology changes

Performance needs

Development Is Iterative:

Understand -> Design -> Build -> Test -> Learn -> Improve -> Repeat

Why Python?

Readable

Beginner-friendly

Widely used

Supports OOP

Strong testing ecosystem

Well supported by AI coding tools

Basic Python Refresher

```
patient_name = 'Alex'
appointment_count = 3
if appointment_count > 0:
    print('Appointments available')
```

Lists and Dictionaries

```
patients = ['Alex', 'Sam', 'Jordan']

patient = {'id': 'P001', 'name': 'Alex'}
```

Functions

```
def register_patient(patient_id, name):
    return {'id': patient_id, 'name': name}
```

Why Functions Matter in Engineering

register_patient()

book_appointment()

cancel_appointment()

search_patient()

Functions support decomposition

First SmartCare Prototype

appointments = []

book_appointment(patient, practitioner, time)

What is missing? Validation? IDs? Status? Conflicts? Persistence? Tests?

What Is Generative AI?

Generates new content from learned patterns

Text

Code

Tests

Explanations

Documentation

Design suggestions

A Simplified View of Large Language Models

Input -> Language Model -> Likely response

An LLM is not a compiler or proof system

Plausible Is Not the Same as Correct

Useful

Incomplete

Outdated

Overcomplicated

Insecure

Wrong

Based on unstated assumptions

Example: Hallucinated Method

if practitioner.is_available(time): ...

Does is_available() exist?

Who defined availability?

What rules determine it?

AI as Tutor

Explain code

Give examples

Clarify concepts

Provide hints

Ask questions that test understanding

Weak Use of AI

Prompt: 'Do my assignment.'

Bypasses learning

Hides assumptions

May create unverifiable work

Leaves the student unable to explain

Better Learning Prompt

Role: Act as a Python tutor

Context: I know functions but am new to SE

Task: Explain why validation matters

Constraint: Do not provide the full solution

Check: Ask me two questions

Human-AI Engineering Loop

Understand -> Specify -> Design -> Generate/Implement -> Verify -> Reflect -> Iterate

Week 1 Practice: Ask - Check - Explain

ASK

CHECK

EXPLAIN

ASK

What am I trying to understand?

What context should I provide?

What should AI not do?

Am I asking for help or replacement?

CHECK

Run the code

Create tests

Consult official documentation

Compare with requirements

Discuss with tutor or peer

Read carefully

EXPLAIN

Can I explain the code?

Can I explain the assumption?

Can I modify it?

Can I justify why it is acceptable?

AI May Assist

Generate

Suggest

Explain

Review

Refactor

Test

Plan

Human Must

Understand

Specify

Judge

Verify

Integrate

Approve

Take responsibility

Meet SmartCare

Fictional community clinic

Spreadsheets and paper

Manual appointment handling

Duplicate bookings

Difficult record lookup

Inconsistent appointment history

Initial Client Request

'We need software to help manage patients, practitioners and appointments.'

Is this enough to start coding?

What Do We Need to Know?

Who uses the software?

What patient information is stored?

How are appointments created?

Can appointments be cancelled?

Should cancelled appointments remain?

Can practitioners be double-booked?

What data must be saved?

SmartCare v0.1 Goal

Problem -> Stakeholders -> Initial features -> Small Python prototype

Not: complete application

Live Demo: Human vs AI

1. Human attempt
2. AI attempt
3. Compare
4. Verify
5. Improve

Evaluate the AI Solution

Does it run?

What assumptions did it make?

What validation is missing?

Is anything out of scope?

Do we understand every line?

How could we test it?

Common AI Failure: Invented Design

doctor.is_available(time)

appointment.confirm()

Where did these methods come from?

Are these requirements confirmed?

Responsible Use Begins Now

Never paste passwords

Never paste API keys or tokens

Avoid confidential source code

Do not paste personal or real patient data

Do not paste restricted assessment content

SmartCare Uses Synthetic Data

Fictional patient names

Fictional practitioner names

Fictional appointment information

No real clinical data

Key Takeaways

Programming is one part of software engineering

SE covers the full lifecycle

AI can assist development

AI output may be wrong or incomplete

Humans verify and take responsibility

Stage 1 Assignment 2 : Rules

Engineering Rule: Understand before generating

AI Rule: AI output is a suggestion until verified

Next Week: From Problems to Requirements

Central question: How do we know what software should actually do?

Stakeholders

Functional and non-functional requirements

User stories

Acceptance criteria